

What an AI SRE is, and what has to be true before you run one

This is the explanation of every idea the workshop uses, written once. You will learn what telemetry an agent can reason over, how a detector decides that something is wrong, what separates a tool an agent may hold from one it may not, and where a person has to stand in the loop. The run sheet carries the timing and the commands for the day, and it points back at these ideas by name.

Read for	engineers and engineering leaders who are deciding how much an agent may do on their systems
Proven by	<code>make check</code> and <code>make e2e</code> in this repository, against a real model and a real browser
Printed by	<code>make book</code> , which writes the bound PDF and reads its own text layer back

Contents

01	An AI SRE is a reader of telemetry that proposes one change Overview
02	An agent can only reason over telemetry that was collected well Module one
03	An error budget turns reliability into a number you can spend Reliability targets
04	A detector decides what is abnormal, and it has to survive a quiet lab Module two
05	Finding what changed is a different question from finding what is wrong Module three
06	What an agent may do is a property of its tools, not of its prompt Module four
07	The human in the loop is an event, not a good intention Module five
08	The risks are specific, and every one of them has a name Module six
09	Where these ideas come from, and what is ours Grounding
10	The terms this workshop uses, in one place Recap

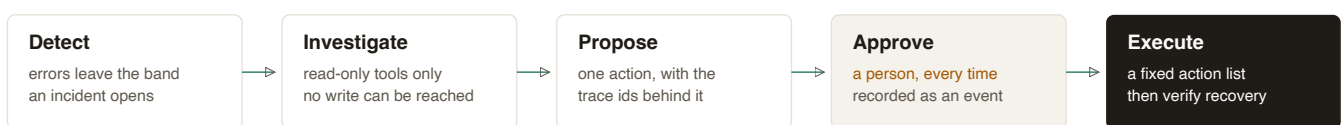
An AI SRE is a reader of telemetry that proposes one change

The job is not to replace the person on call. It is to do the reading that person would do in the first ten minutes, and to arrive with evidence rather than a guess.

The loop runs in five steps, and a person stands in the middle of it loop

An **AI SRE loop** is the path from a signal that something is wrong to a change that fixes it, with each step carrying evidence to the next. A detector notices the signal and opens an incident. An agent investigates with tools that only read. It proposes one action and cites the telemetry it read. A person approves or rejects that action. Only then does anything run, and the system checks that recovery actually happened before it closes the incident.

Each step exists because the one before it can be wrong. A detector fires on noise, so an agent investigates rather than acting. An agent can be confidently wrong, so a person reads the evidence. A person can approve a change that does not work, so the recovery is verified against fresh traffic instead of assumed.



The proposal is refused when a trace id, a service or a release it cites has no telemetry behind it.

The execution is refused when no approval event exists, whatever the agent was told or asked.

Both refusals are the server answering. Neither is a sentence in a prompt.

Figure 1. The loop this lab runs. The two dashed lines are the places it stops on its own, and both are enforced by code rather than by instructions.

The interesting question is not whether a model can read a stack trace. It is where in the loop it acts, what evidence it has to cite, and what stops it when the evidence is not there.

What an AI SRE is not is the part that keeps it safe not

It is not an autonomous operator. It holds no credential that can change production on its own, and the one tool that acts refuses to run without a recorded human approval.

It is not a replacement for alerting. The detector is ordinary statistics over the same telemetry your dashboards read, and it would still be right if no model existed.

It is not a source of truth about your systems. Every claim it makes is checked against the telemetry before the proposal is stored, so a fluent explanation that cites a release nobody deployed is rejected rather than read.

An agent can only reason over telemetry that was collected well

Every failure in this chapter happens before a model is involved. If the three questions below cannot be answered with SQL, no agent will answer them either.

Logs, metrics and traces answer different questions signals

A **log** is a record of one thing that happened, with the detail that was in scope at the time. It answers what went wrong in a single case, and it is the only signal that carries a message a human wrote.

A **metric** is a number aggregated over a window, such as requests per second or the error rate for a service. It answers how much and how often, cheaply, and it is what a threshold is usually written against.

A **trace** is the path of one request across every service it touched, as a tree of spans with timings. It answers where the time went and which hop failed, which neither of the other two can do.

The three are one dataset here. OpenTelemetry is the vendor-neutral standard that produces all of them, and ClickStack stores them in the same database, so a question can start in a log line and end in the trace that produced it.

A wide event beats a narrow one because you cannot add a field later wide-event

A **wide event** is one record per unit of work carrying every attribute you might later want to group by: the release, the region, the customer tier, the endpoint, the outcome.

The cost of a narrow event arrives during an incident. You can see that errors rose and you cannot see that all of them came from one release, because the release was never a field on the record. Nobody adds a field at three in the morning.

This is the first thing to check against your own telemetry: pick any minute of errors and try to split it by release. If you cannot, the rest of the day describes

| something you do not yet have.

Cardinality decides which questions you can afford cardinality

Cardinality is the number of distinct values an attribute takes. A status code has a handful. A user id has as many as you have users.

High cardinality is what makes an attribute useful for finding the one broken thing, and it is also what makes a metrics system expensive, because a traditional one stores a separate series per combination of values. A columnar store reads them as columns instead, which is why the queries in this lab can group by anything without a per-series cost.

Sampling removes exactly the rows an investigation needs sampling

Sampling keeps a fraction of traces to control cost. The fraction is chosen when everything is healthy, and the rows it discards are ordinary ones.

The failure is specific rather than general. A rare error path is rare, so sampling is most likely to drop the requests that failed, and a percentile computed from what survives is a percentile of the requests that worked. This lab keeps every span, which is why a latency percentile here is the real one.

An error budget turns reliability into a number you can spend

These three terms come from Google's site reliability engineering practice, and they are the vocabulary the rest of the day uses. They are worth stating precisely because most alerting arguments are really disagreements about them.

An indicator measures what a user experiences sli

A **service level indicator** is a measurement of one thing a user can feel: the proportion of requests that succeeded, or the proportion served faster than some latency.

It is written as good events over valid events, which keeps it a ratio rather than a count, so it stays comparable when traffic changes.

An objective is the target the indicator has to hold slo

A **service level objective** is the threshold an indicator must stay above over a stated window, such as 99.9 percent of signups succeeding over thirty days.

The window matters as much as the number. An objective with no window cannot be measured, and an objective nobody would act on when it is missed is decoration.

The budget is the failure the objective already allows error-budget

An **error budget** is the difference between perfect and the objective, expressed as an amount of failure you are permitted to spend. An objective of 99.9 percent over thirty days is a budget of about forty three minutes.

It reframes the argument. Reliability stops being a value people hold and becomes a quantity that a release either spends or does not, which is a conversation an engineering team and a product team can actually have.

Burn rate is what makes an alert worth waking somebody for `burn-rate`

Burn rate is how fast the error budget is being consumed relative to the objective window. A rate of one exhausts the budget exactly at the end of the window, and a rate of two exhausts it in half the time.

Alerting on burn rate rather than on a raw threshold is what separates a page that means something from one that does not. A short window catches a sudden spike, a longer window confirms that it is sustained, and requiring both is how the same rule avoids paging on a blip.

The lab's detector is a simpler relative of this, because a workshop needs an incident within about a minute. The reasoning is the same: compare against what normal looks like, not against a number somebody typed once.

A detector decides what is abnormal, and it has to survive a quiet lab

Detection is arithmetic over a baseline. The difficulty is never the arithmetic; it is choosing what counts as normal when the system has been broken recently.

A baseline is a claim about what normal looked like `baseline`

A **baseline** is the distribution of a signal over a recent period that you are willing to call healthy. Everything a detector says is relative to it.

Choosing the period is the whole design. Too short and a slow morning reads as an outage; too long and a service that has been degrading for a week looks fine, because the degradation is now part of normal.

Three sigma is a band around the baseline, not a truth `three-sigma`

A **three sigma bound** is the mean of the baseline plus three times its standard deviation. A minute above it is unusual enough to be worth opening an incident for.

It assumes the baseline is roughly bell shaped, which error rates often are not, so the bound is a useful trigger rather than a statement about probability. That is why this lab pairs it with a fixed ceiling: when there is no clean baseline yet, an error rate above a flat percentage opens an incident anyway.

A baseline that includes the last incident hides the next one `incident-exclusion`

When an incident window is left in the baseline, the mean and the deviation both rise, and the band widens to include the failure you just had.

The cost is exact and it shows up in a workshop within the hour. Run one chaos scenario, then run a second, and the second does not open an incident because the first one taught the detector that this is normal. The detector here excludes past incident windows from its baseline for that reason.

Alert on what a user feels, not on what a machine is doing symptom-alerting

Symptom based alerting pages on user visible failure, such as signups returning errors. Cause based alerting pages on machine conditions, such as processor use above eighty percent.

The second kind is where alert fatigue comes from. A cause that does not reach a user is a page that did not need to happen, and a team that has been woken by enough of those learns to ignore the next one, which is the failure that matters.

Finding what changed is a different question from finding what is wrong

By this point you know something is failing. Everything here is about narrowing from the whole system to the one attribute that separates the failing requests from the working ones.

An event delta ranks the attributes that differ between two windows event-delta

An **event delta** compares the failing window against a healthy one and ranks every attribute by how differently it is distributed between them.

It answers the question an engineer asks first and phrases badly: what is different about the broken ones? When one release, one endpoint or one region appears at the top, the investigation is over in a step rather than a shift.

Clustering log lines turns thousands of messages into a handful log-pattern

Log patterns group messages that share a template and differ only in their variables, so ten thousand lines collapse into a short list with counts.

What you are looking for is a pattern that is new, rather than one that is large. The largest cluster during an incident is usually the same noise as during a healthy hour.

A trace waterfall shows where the time actually went trace-graph

A **trace waterfall** lays out the spans of one request against time, so a slow parent with one slow child is visibly different from a slow parent with fifty fast ones.

It also exposes the case that has no other signal. A request that hangs and never returns exports no completed span at all, so it is missing from the trace data and

visible only as logs with a trace id and no waterfall to match.

That asymmetry is worth teaching directly. Absence is evidence here, and an agent that only queries for spans that exist will never find a hang.

What an agent may do is a property of its tools, not of its prompt

This is the chapter that decides whether any of the rest is safe to run on a real system. A sentence in a prompt is a request. A tool that does not exist is a limit.

Every tool is classified before it can be registered tool-class

A **tool class** states what a tool is allowed to do: read telemetry, store a proposal, or act on a system. The classification lives in code next to the server, and a tool with no class cannot be registered at all.

The structure is what makes this reviewable. Somebody adding a tool cannot forget to think about its class, because the process refuses to start when they do.

The database user is read-only, so raw SQL is still bounded least-privilege

Least privilege means the credential an agent holds cannot perform the action you are relying on it not to perform.

Giving a model a SQL tool sounds alarming until the account it runs as is read-only, capped in execution time, capped in memory and capped in result rows. Then the worst case of a badly written query is a query that fails, rather than an ingest pipeline starved behind it.

A proposal that cites telemetry nobody can find is refused grounding

Grounding is the check that every identifier in a proposal resolves to something real: the trace ids exist, the service exists, the release was deployed.

This is the guard against the failure mode that matters most in operations. A model can produce a fluent, plausible root cause for an incident it has not

understood, and the sentence reads exactly like one it has. Deterministic code checking the citations is what separates the two, and the check runs before the proposal is stored rather than after a person has read it.

An action is chosen from a list, never assembled from text `action-list`

The remediation runner takes a named action and its arguments from a fixed list, and builds the command itself.

The alternative is the vulnerability. A runner that executes a command the model composed turns every log line the model read into a potential instruction, because an attacker who can write to your logs can write to your agent's context.

The human in the loop is an event, not a good intention

Everybody agrees a person should approve production changes. The question this chapter answers is what that sentence has to become in code before it is true.

Approval is a stored event the execution path reads `approval-event`

An **approval event** is a row written when a named person approved a specific proposal. The tool that acts looks for that row and refuses without it.

Ask the agent to execute anyway and it is refused, and the refusal is recorded as well. That is the difference between a guardrail and a request: the server answers, and the answer does not depend on what the model was asked.

The audit log is append-only, so state is derived rather than edited `audit-log`

An **append-only audit log** records every step as a new row, and the current state of an incident is computed from the rows rather than stored beside them.

A row that can be updated is a history that can disagree with itself. Deriving the state means the log and the incident cannot drift apart, and it means the question of who approved what has one answer.

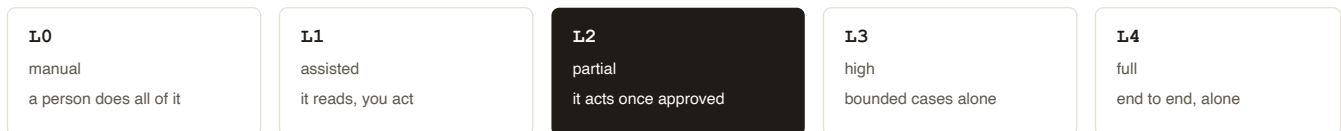
An incident closes on fresh traffic, not on a successful command `recovery-check`

A **recovery check** watches the same signal that opened the incident for a period after the action ran, and only then resolves it.

A rollback command that exits zero has proved that a command ran. It has not proved that signups work, and those are different claims. This is the same rule the engineering standard applies to tests: verify by executing, never by reading.

Autonomy is a level you choose per action, not a switch `autonomy-level`

Google's site reliability engineering practice describes five **autonomy levels** for operations, from fully manual to fully autonomous, and the useful move is to place each action on the ladder separately rather than to place the whole agent.



Moving one action to L3 is a decision about blast radius: how much damage it can do, and how quickly it can be undone.

Figure 2. The ladder, with the lab's position marked. A rollback of a release you deployed an hour ago and a change to a database are not the same rung, and nothing forces them to share one.

The honest version of the question is not whether you trust the model. It is which actions you would let a new engineer run unsupervised in their first week, and why.

The risks are specific, and every one of them has a name

OWASP published a list of the threats that are particular to agents, rather than to models, in December 2025. Four of them apply directly to anything that reads production telemetry and can act on it.

Log lines are attacker-controlled text that your agent reads prompt-injection

A **goal hijack** is an instruction placed where an agent will read it, causing it to pursue somebody else's objective. OWASP lists it first among the agentic risks, because agents cannot reliably tell instructions from content.

Telemetry is the ideal delivery route. Anyone who can make your system log a string they chose, such as a username or a user agent, can put a sentence in front of your agent. The defence is not a better instruction: it is that the acting tool takes a named action from a list and never a command from text.

What an agent remembers is an attack surface too context-poisoning

Memory and context poisoning is the same idea applied over time: something false is written into the material an agent carries between steps, and every later step reasons from it.

An incident summary that an agent wrote and will read again is memory. Deriving state from an append-only log rather than from a summary is what keeps a wrong sentence from becoming the record.

An approval nobody could disprove in a minute is a formality approval-fatigue

Trust exploitation is OWASP's name for the risk that a person approves because approving is what they do. A prompt that offers only Approve and Reject at four in the morning is a formality with a button.

What makes an approval real is what sits on the page: the evidence the proposal cites, the blast radius of the action, and a link to the telemetry that supports it. The question to ask of your own approval page is what a reader could actually disprove in a minute.

Privilege you granted for convenience is privilege an agent has excessive-agency

Privilege abuse and **tool misuse** are the two risks that follow from a capability being wider than the job. A container runtime socket mounted so that one action can restart one service is a socket that can start anything.

Google's operations guidance names the same controls from the other direction: no ambient access, least privilege, a circuit breaker that stops a run that is going wrong, and a dry run that shows what would happen before anything does.

This lab is a workshop and it makes that trade deliberately, which is why the hardening module exists. Copying the architecture without the hardening is the move that goes wrong.

An agent with no test set drifts without anybody noticing eval-set

An **evaluation set** is a collection of past incidents with known answers, replayed against the agent so that a change in prompt, model or tool is measured rather than felt.

Without one, the only signal is impressions during incidents, which is the worst moment to be forming them. Keep every incident you resolve, with what the cause turned out to be, and you have the beginning of one.

Where these ideas come from, and what is ours

None of the vocabulary in this book was invented here.

Knowing which part is published practice and which part is one lab's choice is what lets you argue with the second kind.

The reliability vocabulary is Google's, and it predates agents published

Indicator, objective and error budget are defined in Implementing SLOs (<https://sre.google/workbook/implementing-slos/>), chapter two of Google's Site Reliability Workbook. Burn rate and the multiwindow, multi-burn-rate approach come from Alerting on SLOs (<https://sre.google/workbook/alerting-on-slos/>), chapter five, which works through six alerting strategies and recommends the last of them.

The signals themselves are a standard rather than a product. OpenTelemetry (<https://opentelemetry.io/docs/concepts/signals/>) defines traces, metrics and logs, which is why the queries in this lab describe a shape you can reproduce on a different vendor's storage.

The guardrails have names, and a ladder to place them on guardrail-source

Google's own guidance on AI engineering for reliable operations (<https://sre.google/resources/practices-and-processes/ai-engineering-reliable-operations/>) names the controls this lab implements: no ambient access and least privilege, circuit breakers that stop a run going wrong, dry run support, and actuation that is safe by default. It also gives the five autonomy levels used in the human chapter.

The mapping is close enough to be worth checking yourself. Read their list, then read what this lab enforces, and the gap you find is the work you would still have to do before running any of it on your own systems.

The risks are catalogued, so you can argue about them precisely risk-source

OWASP published its Top 10 for Agentic Applications (<https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>) in December 2025, covering the threats that belong to agents rather than to models. Five of them are the hardening chapter: goal hijack, tool misuse, privilege abuse, memory and context poisoning, and trust exploitation.

Using their names is not ceremony. It means a security reviewer at your company can map this architecture onto a list they already hold, instead of reading an argument written from first principles by a workshop.

What is this lab's own choice, and therefore arguable ours

The three sigma bound, the fixed ceiling underneath it, the decision to exclude past incident windows from the baseline, and the choice to hold every action at the approved rung are ours. They are defensible and they are not standards.

Treat them the way you would treat any other design in a repository you did not write. The published material tells you what the options are; the lab tells you what one set of answers costs.

Recap

The terms this workshop uses, in one place

If you take one page away, take this one. Each term links to the explanation it came from.

Term	What it means here
Wide event	One record per unit of work, carrying every attribute you might later group by
Cardinality	How many distinct values an attribute has, which decides what a query costs
Sampling	Keeping a fraction of traces, which removes the rare rows an investigation needs
Indicator	A measurement of something a user can feel, as good events over valid events
Objective	The threshold that indicator must hold, over a stated window
Error budget	The failure the objective already allows, expressed as an amount to spend
Burn rate	How fast the budget is being spent, relative to the window
Baseline	The period you are willing to call healthy, which everything else is relative to
Three sigma	A band around the baseline, wide enough that ordinary minutes stay inside it
Event delta	The attributes that are distributed differently in a failing window
Trace waterfall	The spans of one request against time, which shows where the time went
Tool class	What a tool may do, declared in code, without which it cannot register
Grounding	Checking that every identifier a proposal cites resolves to real telemetry
Approval event	The stored record of a person agreeing, which the acting tool reads
Recovery check	Watching the original signal on fresh traffic before closing an incident

Term	What it means here
Autonomy level	How much an action runs unattended, chosen per action rather than per agent
Goal hijack	An instruction planted where an agent will read it, such as a log line
Evaluation set	Past incidents with known answers, replayed so a change is measured

The run sheet for the day is `teach.html`, and it cites these ideas by name rather than explaining them again. Everything described here is exercised by `make e2e` against a real model, and scored by `make score`.